

VIET NAM NATIONAL UNIVERSITY – HO CHI MINH
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY



MANGAHUB – MANGA & COMIC TRACKING SYSTEM

Course: Net-Centric Programming – IT096IU

Instructors: Lê Thanh Sơn

Programming Language: Go (GoLang)

Group members:

Trần Khánh Tài – ITITIU21300

Nguyễn Dy Niên – ITITIU21272

Academic Year: 2025 – 2026

Contents

Abstract	5
1. Introduction	6
1.1. Background and Motivation	6
1.2. Problem Statement	6
1.3. Objectives of the System	6
1.4. Scope of the Project	6
2. Theoretical Background.....	7
2.1. Go (GoLang) and Concurrency	7
2.2. Network Protocols	7
2.3. Containerization with Docker and CI Foundation.....	7
3. System Analysis	8
3.1. Functional Requirements	8
3.2. Non-Functional Requirements	8
4. System Design.....	8
4.1. Overall System Architecture	8
4.2. Database Design (ERD)	9
4.2.1. Entity Definition	9
4.2.2. Relationship Analysis.....	11
4.3. API & Protocol Design.....	11
4.3.1. RESTful API Design (HTTP/1.1).....	12
4.3.2. TCP Stateful Synchronization Protocol	13
4.3.3. Reliable UDP (RUDP) Notification System	14
4.3.4. WebSocket Protocol (Real-time Communication).....	16
4.3.5. gRPC Service (Internal Orchestration).....	17
5. Implementation	18
5.1. Development Environment	18
5.2. Persistence Layer Implementation	20
5.2.1. Schema Initialization and Migration	20
5.2.2. The “Upsert” Pattern for Real-time Sync	20
5.3. Network Protocol Implementations	21
5.3.1. HTTP API & JWT Middleware	21
5.3.2. TCP Progress Synchronization.....	23
5.3.3. Reliable UDP Notifications (RUDP).....	25
5.3.4. WebSocket Chat (Actor Model).....	28
5.3.5. gRPC Microservice Implementation.....	30
6. Results and Evaluation.....	32

6.1.	System Performance & Concurrency	32
6.2.	Architecture Evaluation.....	33
6.3.	Limitations.....	34
7.	Conclusion and Future Work.....	35
7.1.	Conclusion	35
7.2.	Future Enhancements.....	35
8.	References	35

Table of Figure

Figure 1 All containers running	8
Figure 2 System Architecture Diagram	9
Figure 3 ERD Diagram	11
Figure 4 Manga Search & Filtering Flow	13
Figure 5 Update Progress Workflow	14
Figure 6 UDP Notification Workflow	15
Figure 7 Real-time Chat Message Broadcast	16
Figure 8 Internal gRPC Communication Flow	17
Figure 9 Running 3 Client at the same time	33
Figure 10 Chatting with each other via WebSocket	33
Figure 11 Resilience Test	34

Abstract

The rapid growth of digital comic consumption necessitates platforms capable of real-time synchronization, high concurrency, and distributed data management. This paper presents the design and implementation of MangaHub, a comprehensive manga tracking system built entirely in Go. The system architecture demonstrates the practical application of five distinct network protocols—HTTP, TCP, UDP, WebSocket, and gRPC—each selected to solve specific distributed computing challenges. By leveraging Go's native concurrency model (goroutines and channels), reliable UDP delivery confirmation (RUDP), and containerization via Docker, MangaHub achieves reliable progress tracking, real-time communication, and secure internal microservice orchestration.

1. Introduction

1.1. Background and Motivation

The rapid digital transformation in the entertainment industry has led to a massive increase in digital comic and manga consumption. However, managing reading progress, discovering new content, and discussing with other readers across the multiple platforms remain fragmented experiences. Furthermore, from a software engineering perspective, building a real-time, distributed system that handles multiple types of network traffic (RESTful, real-time streams, internal microservices) is a complex challenge. This project, MangaHub, is motivated by the need to build a comprehensive manga tracking platform while applying advanced Net-Centric programming concepts.

1.2. Problem Statement

Current manga tracking systems often rely heavily on traditional HTTP polling, which is inefficient for real-time notifications and live chat. Additionally, handling concurrent reading progress updates from multiple devices requires robust synchronization mechanisms. The challenge is to design a cohesive system that effectively utilizes different network protocols (TCP, UDP, HTTP, WebSocket, gRPC) for their most appropriate use cases without compromising system stability.

1.3. Objectives of the System

The main objective of this project is to design and develop a network-centric Manga Tracking System using Go. The specific goals include:

- Implementing all five core communication protocols: HTTP (REST API), TCP (Progress Sync), UDP (Notifications), WebSocket (Real-time Chat), and gRPC (Internal Services).
- Providing role-based access control with JWT authentication.
- Supporting advanced user features such as Multiple Reading Lists and real-time chapter release notifications with UDP Delivery Confirmation (RUDP).

1.4. Scope of the Project

This project focuses on the back-end architecture and network communication of the MangaHub System. It includes user authentication, manga catalog management, progress

synchronization, and a real-time chat room. The application is containerized using Docker and utilizes SQLite for data persistence.

2. Theoretical Background

2.1. Go (GoLang) and Concurrency

Go is a statically typed, compiled programming language designed at Google. It provides built-in concurrency mechanisms, notably goroutines and channels, which make it highly efficient for network programming. In MangaHub, goroutines are extensively used to handle concurrent client connections across TCP, UDP, and WebSocket servers.

2.2. Network Protocols

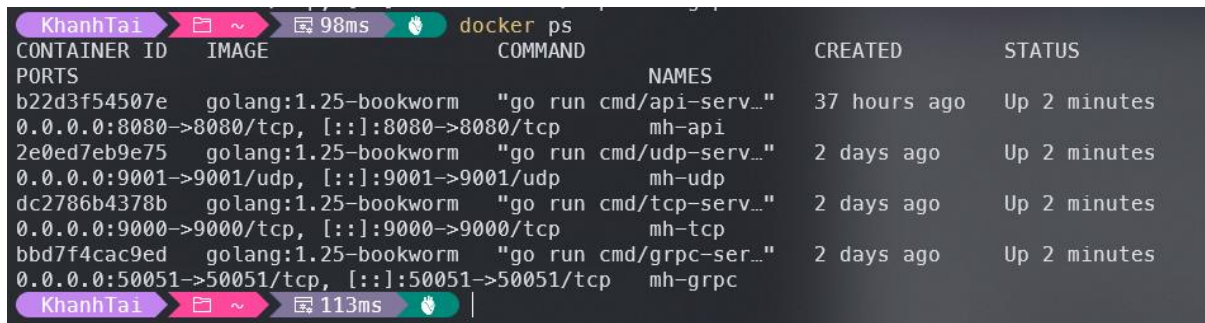
- **HTTP/REST:** Used for standard CRUD operations and client-server interactions (e.g., login, fetching manga lists).
- **TCP (Transmission Control Protocol):** A connection-oriented protocol used in this system for reliable, stateful synchronization of reading progress.
- **UDP (User Datagram Protocol):** A connectionless protocol utilized for broadcasting chapter release notifications.
- **WebSocket:** A full-duplex communication protocol over a single TCP connection, ideal for the system's real-time chat rooms.
- **gRPC & Protocol Buffers:** A high-performance Remote Procedure Call (RPC) framework using binary serialization (Protobuf), used for internal microservice communication.

2.3. Containerization with Docker and CI Foundation

The project utilizes **Docker** and **Docker Compose** to ensure environment consistency.

- **Dockerization:** Every service (api-server, tcp-server, udp-server, grpc-server) is containerized using Go-based image. This allows the system to be scaled independently.
- **Continuous Integration (CI) Logic:** While the current setup focuses in deployment, the architecture is CI-ready. A standard pipeline would involve:
 - **Build Stage:** Compiling Go binaries.
 - **Test Stage:** Running unit tests (e.g., in internal/grpc/service_test.go).

- **Docker Stage:** Building images and pushing to registry.



CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS
b22d3f54507e	golang:1.25-bookworm	"go run cmd/api-serv..."	mh-api	37 hours ago	Up 2 minutes
2e0ed7eb9e75	golang:1.25-bookworm	"go run cmd/udp-serv..."	mh-udp	2 days ago	Up 2 minutes
dc2786b4378b	golang:1.25-bookworm	"go run cmd/tcp-serv..."	mh-tcp	2 days ago	Up 2 minutes
b2d7f4cac9ed	golang:1.25-bookworm	"go run cmd/grpc-ser..."	mh-grpc	2 days ago	Up 2 minutes

Figure 1 All containers running

3. System Analysis

3.1.Functional Requirements

- **Authentication:** Users must be able to register and log in securely using JWT.
- **Manga Discovery:** Users can search and filter manga by genres, authors, and status.
- **Progress Tracking:** The system must sync reading progress via TCP and update the database reliably.
- **Real-time Chat:** Users can join manga-specific chat rooms via WebSockets.
- **Notifications:** The system must broadcast chapter releases via UDP, implementing an acknowledgment (ACK) mechanism for reliability.

3.2. Non-Functional Requirements

- **Concurrency:** The system must handle 50-100 concurrent users seamlessly.
- **Reliability:** The UDP server must implement a retry mechanism for unacknowledged notifications.
- **Modularity:** The system must be divided into independent servers (API, TCP, UDP, gRPC)

4. System Design

4.1. Overall System Architecture

MangaHub adopts a Distributed Service Architecture. The system is divided into four distinct Go servers running simultaneously via Docker Compose:

- api-server (Port 8080): Handles HTTP REST requests and upgrades WebSocket connections.
- tcp-server (Port 9000): Manages continuous TCP sockets for reading progress sync.
- udp-server (Port 9001): Broadcasts notifications and tracks client ACKs.
- grpc-server (Port 50051): Serves internal data requests (e.g., Profile fetching, Search) using Protobuf.

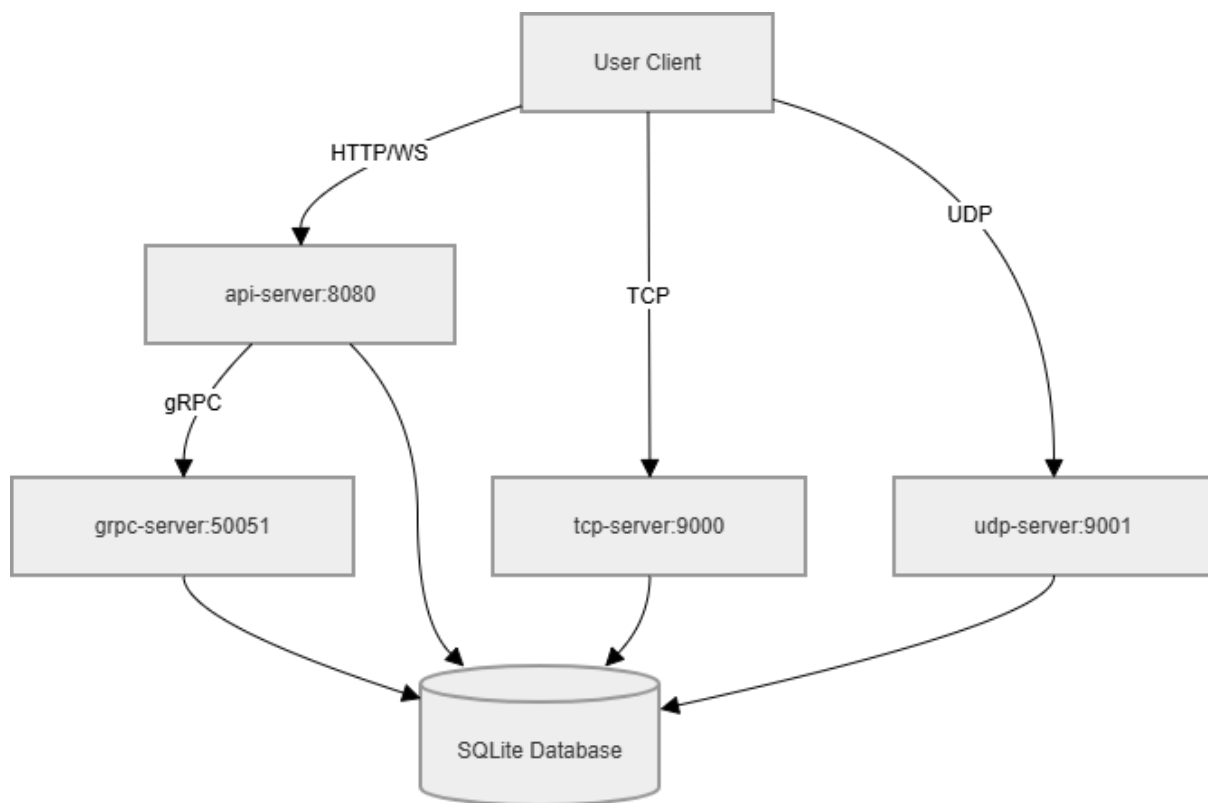


Figure 2 System Architecture Diagram

4.2. Database Design (ERD)

The persistence layer of MangaHub is built on **SQLite**, a serverless, zero-configuration database engine that is ideal for containerized microservices due to its portability and efficiency. To ensure data consistency and eliminate redundancy, the schema is strictly normalized to the **Third Normal Form (3NF)**. **users:** Stores user credentials and hashed passwords.

4.2.1. Entity Definition

- **users:** This entity serves as the primary identity store for the system. It contains unique identifiers (`id`) and stores `username` with a `UNIQUE` constraint to prevent duplicate registrations. Security is prioritized by storing only the `password_hash` (generated via Bcrypt) rather than plain-text credentials.
- **manga:** Acts as the central catalog for the application. It stores comprehensive metadata including `title`, `author`, `genres` (stored as a serialized string), `total_chapters`, and descriptive content. Recent migrations have also added a `cover_url` field to support front-end visualization.
- **user_lists:** This table facilitates the "Multiple Reading Lists" feature. It allows users to create custom categories (e.g., "Reading", "Plan to Read", "Favorites"). A unique constraint is placed on the combination of `user_id` and `name` to ensure a user cannot have two lists with the identical name.
- **user_list_items:** A complex associative entity that links users, their custom lists, and specific manga titles. This table is the "heart" of the tracking system, maintaining the `current_chapter` and specific reading status for every manga within a user's library. It utilizes a composite primary key (`user_id`, `list_id`, `manga_id`) for optimized indexing.
- **user_progress:** Designed for high-frequency updates, particularly for the TCP and gRPC synchronization services. It maintains a direct relationship between a user and a manga title, allowing for rapid lookups and updates of the `current_chapter` without traversing the custom list hierarchy.
- **user_notification_prefs:** This table manages the configuration for the Reliable UDP (RUDP) notification system. It stores user-specific preferences in a `prefs_json` format, enabling the server to determine which chapter releases or system alerts should be broadcasted to a specific client address.

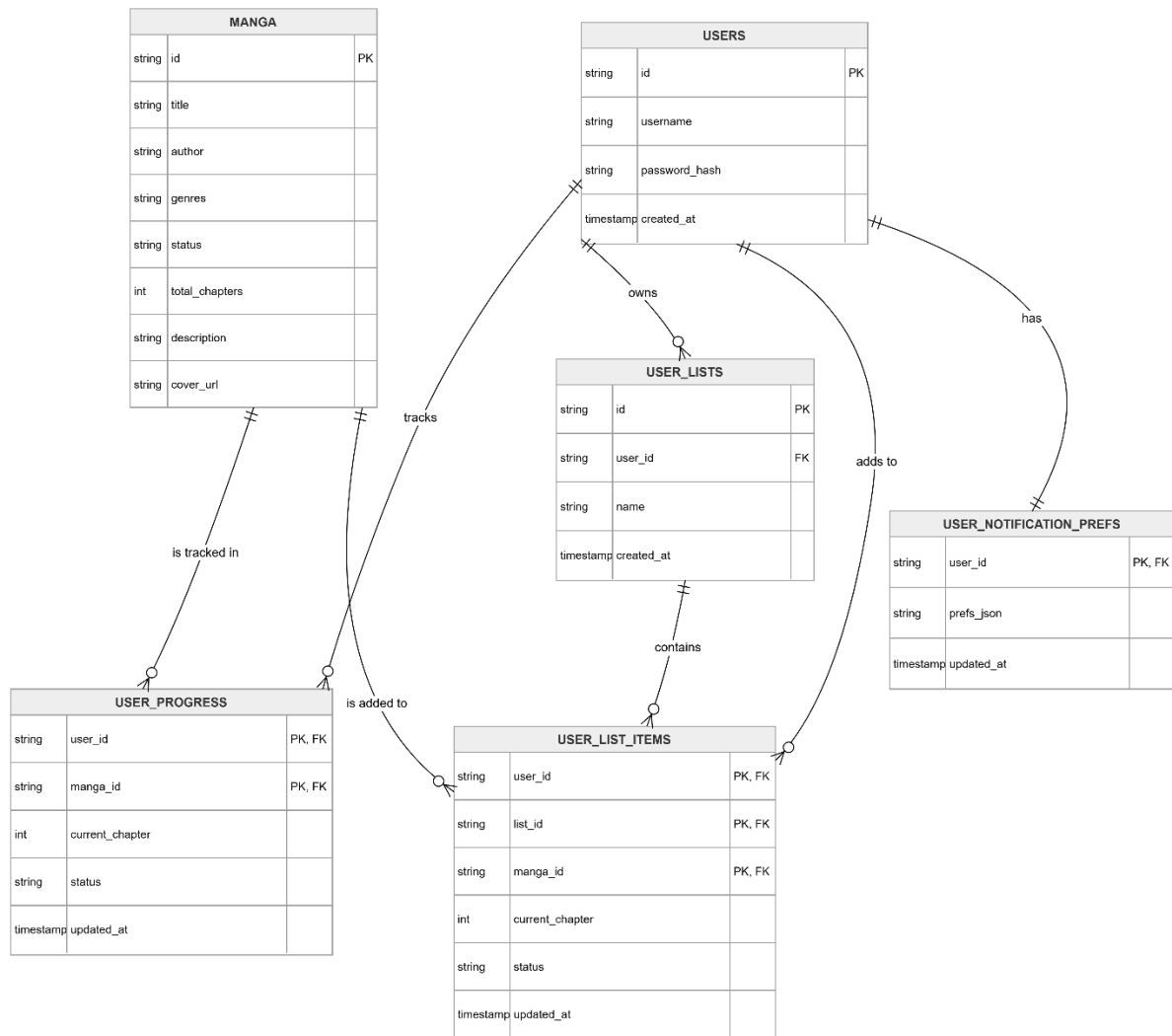


Figure 3 ERD Diagram

4.2.2. Relationship Analysis

The system architecture relies heavily on **Foreign Key constraints** with ON DELETE CASCADE actions to maintain referential integrity.

- **One-to-Many (1:N)**: A single user can own multiple user_lists.
- **Many-to-Many (M:N)**: The user_list_items table resolves the complex relationship between users, lists, and manga, allowing a single manga title to appear in different lists for different users.

4.3. API & Protocol Design

This section outlines the architectural decisions regarding the communication interfaces and data exchange standards. MangaHub adopts a multi-protocol strategy, where each protocol is selected to optimize specific distributed computing requirements, such as reliability, low latency, or high-performance internal orchestration.

4.3.1. RESTful API Design (HTTP/1.1)

The HTTP REST API serves as the primary gateway for stateless client-server interactions, primarily handling user onboarding and metadata retrieval.

- **Design Principles:** The API follows standard REST principles, utilizing HTTP methods (GET, POST, PUT) to manage resources.
- **Authentication Design:** A token-based authentication mechanism using JSON Web Tokens (JWT) is designed to ensure secure access to protected resources without maintaining session state on the server.
- **Key Interface Definitions:**
- **Identity Management:** Endpoints such as /auth/register and /auth/login handle user credentials and token issuance.
- **Resource Retrieval:** The /manga endpoint is designed with query parameter support for searching and filtering (by genre, author, and title).

Method	Endpoint	Description	Request Parameters / Body
GET	/health	System health check and heartbeat.	None
POST	/auth/register	Registering a new user account.	Body: username, password (min 8 character)
POST	/auth/login	Authenticates users and responds to a token.	Body: username, password
GET	/manga	Search for manga with complex filters and pagination	Query: title, author, genre, limit, offset
GET	/manga/:id	Fetches full metadata for a specific manga title.	Path: id
POST	/users/library	Adds a manga to the user's reading list.	Body: manga_id, list_name, status
PUT	/users/progress	Atomically updates the reading chapter and status.	Body: manga_id, current_chapter, status

GET	/ws/chat	Establishes a WebSocket connection for real-time chat.	Query: token (JWT), room
-----	----------	--	--------------------------

Table 1: API Endpoint Table

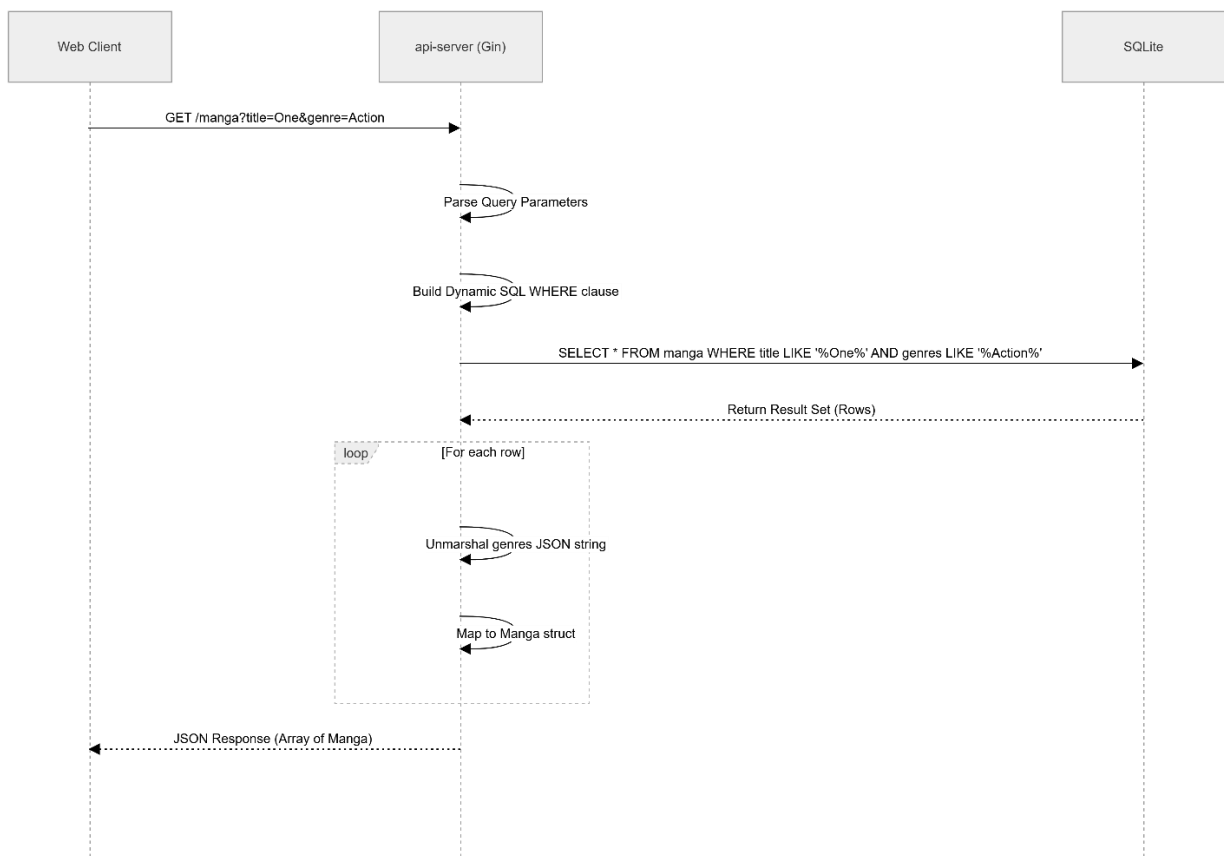


Figure 4 Manga Search & Filtering Flow

The search protocol leverages HTTP Query Parameters to allow flexible filtering. The server dynamically constructs SQL queries to ensure efficient database lookups, then transforms the raw database rows (including JSON strings for genres) into structured JSON responses for the client.

4.3.2. TCP Stateful Synchronization Protocol

For reading progress synchronization, a persistent, stateful protocol over TCP is designed to ensure ordered and reliable delivery of updates.

- **Rationale:** TCP is selected for its connection-oriented nature, which is critical for maintaining a "live" session of a user's reading activity across multiple devices.

- **Handshake & Framing:** The design requires a two-stage handshake: a standard TCP connection followed by an application-level auth JSON frame.
- **Message Schema:** Messages are encapsulated in newline-delimited JSON frames. This approach allows for flexible payload structures, including ping/pong for heartbeat and progress for chapter updates.

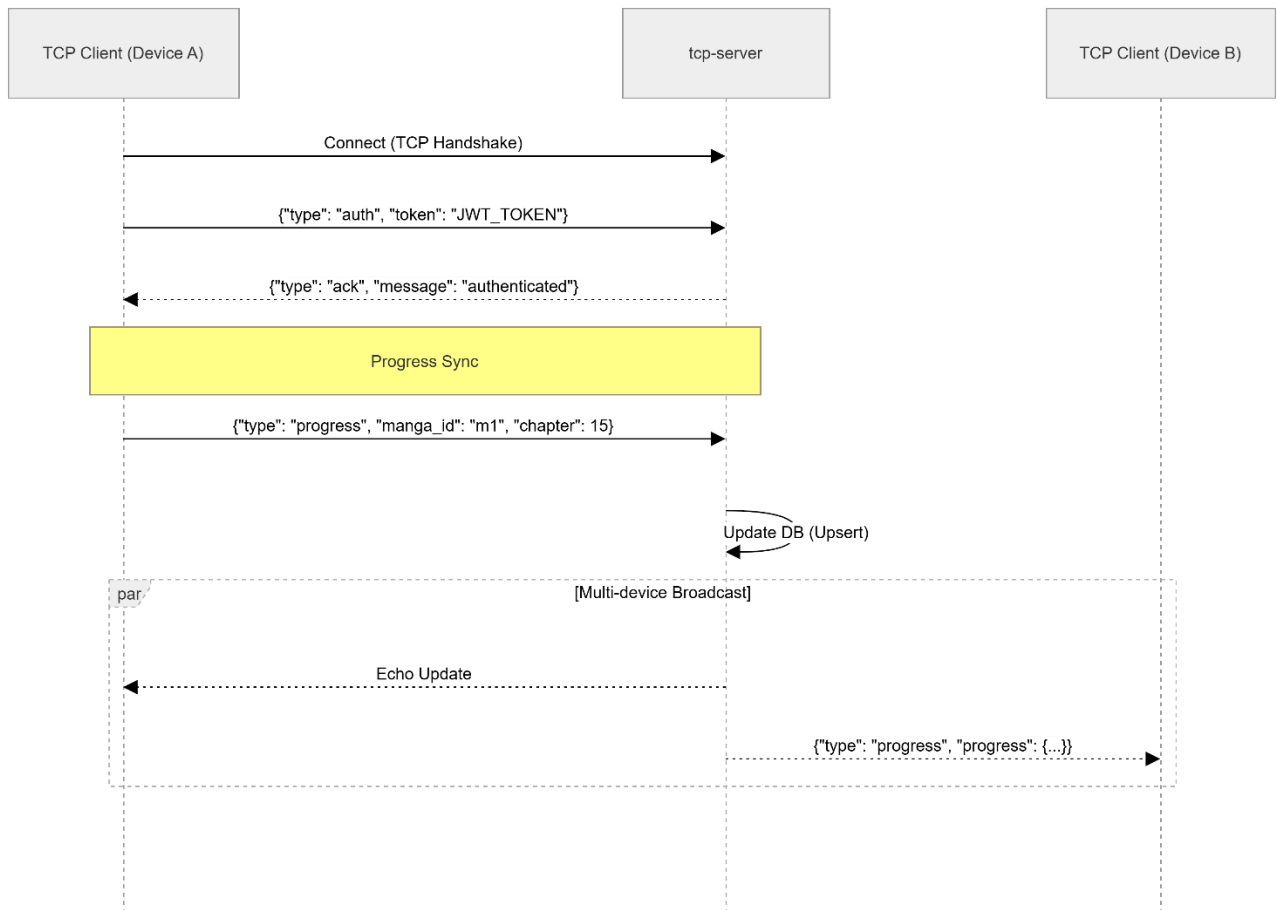


Figure 5 Update Progress Workflow

4.3.3. Reliable UDP (RUDP) Notification System

To support real-time broadcasting of manga updates with low network overhead, the system incorporates an application-level acknowledgment (ACK) mechanism over UDP.

- **Reliability Design:** Unlike standard UDP, this protocol expects an ACK message for every notification broadcast. The server maintains a "Pending ACK" state for each active client.

- **Retry Logic:** An exponential backoff or fixed-interval retry strategy is designed to re-transmit packets if an ACK is not received within a specified timeout (e.g., 3 seconds).

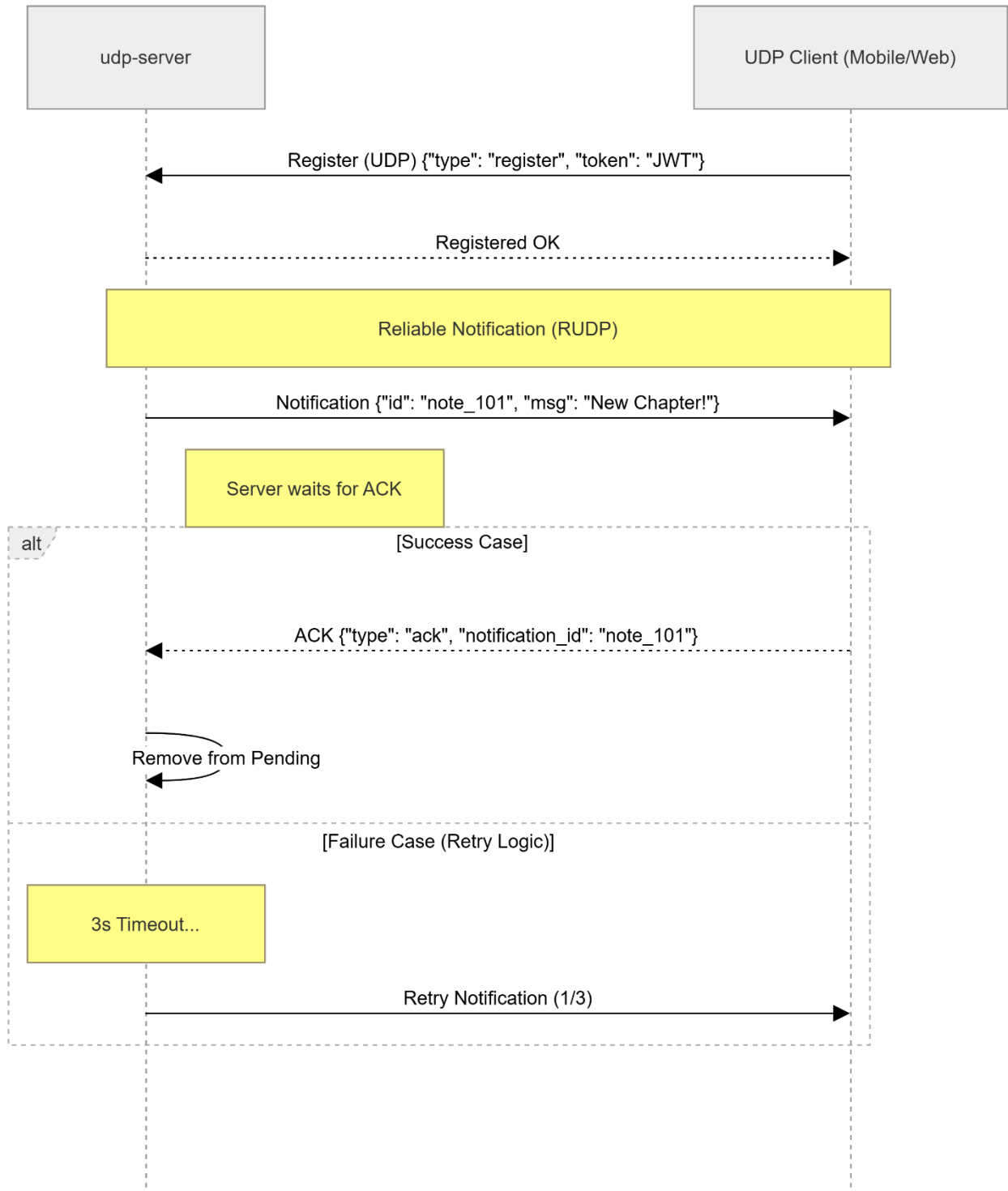


Figure 6 UDP Notification Workflow

4.3.4. WebSocket Protocol (Real-time Communication)

To support the community aspect of MangaHub, the system utilizes the WebSocket protocol for full-duplex, real-time messaging.

- **Protocol Upgrade:** Connections are initially established via HTTP at /ws/chat and then upgraded to a persistent WebSocket connection using the **Gorilla WebSocket** library.
- **Payload Structure:** Communication relies on JSON payloads. A standard message includes a "message" string, while the server automatically injects context such as the user_id and username derived from the JWT, ensuring that messages cannot be spoofed.
- **Room Isolation:** The protocol design includes a "room" or "manga_id" parameter in the connection query, allowing the **ChatHub** to isolate conversations to specific manga titles, preventing message overflow across the entire platform.

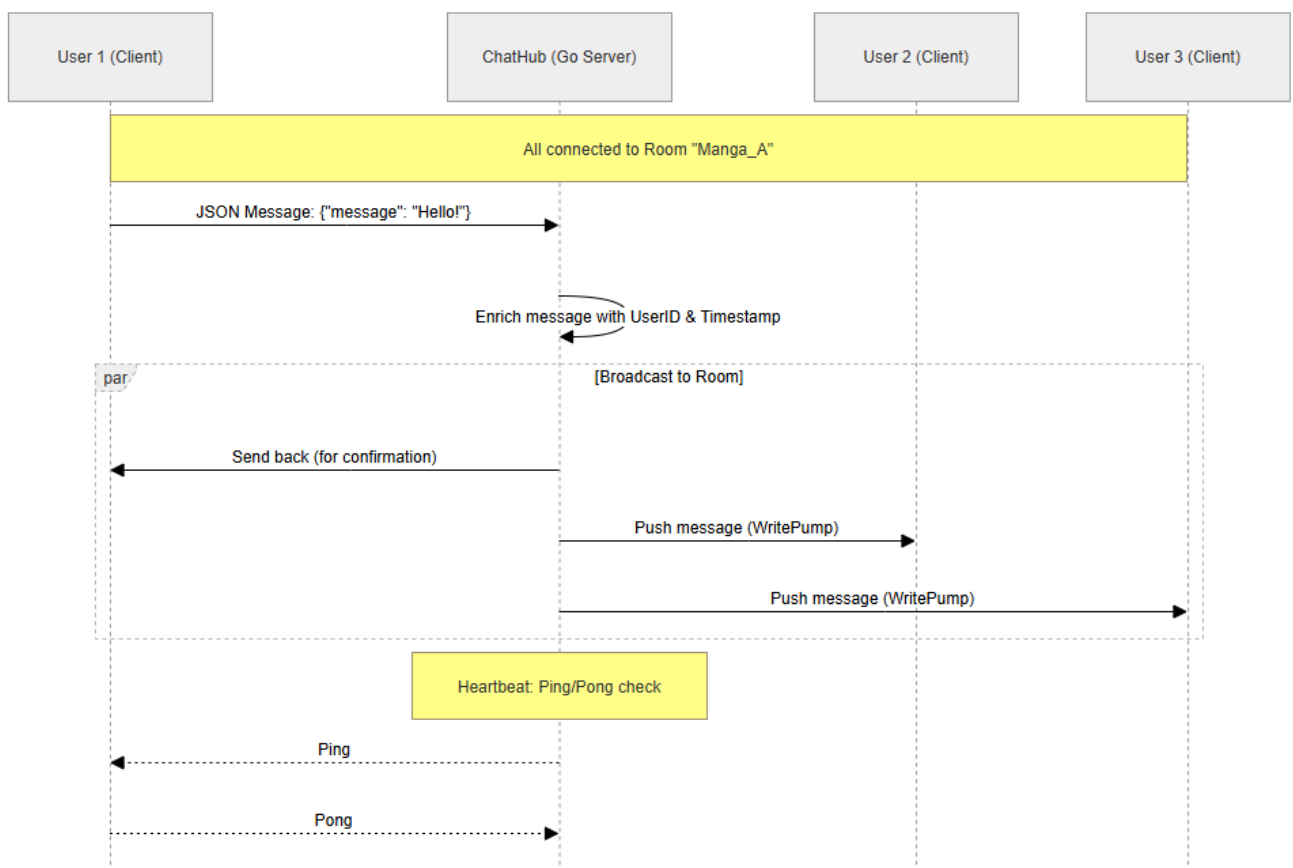


Figure 7 Real-time Chat Message Broadcast

The WebSocket implementation uses a centralized ChatHub following the Actor Model. Messages are received, enriched with server-side metadata to prevent spoofing, and then

fanned out concurrently to all active connections mapped to the specific manga_id or room.

4.3.5. gRPC Service (Internal Orchestration)

For high-performance, internal microservice communication, MangaHub implements **gRPC** with **Protocol Buffers (Protobuf)** as the Interface Definition Language (IDL).

- **Efficiency:** Unlike REST/JSON which uses text-based serialization, gRPC uses binary serialization via Protobuf, significantly reducing the payload size and improving parsing speed.
- **Service Definition:** The MangaService and UserService are defined in mangahub.proto. This allows the server to expose methods like GetManga for retrieving detailed metadata and UpdateProgress for cross-service data synchronization.
- **Metadata Auth:** Security in gRPC is handled via **Metadata**, where the JWT is passed along with the RPC call, allowing the gRPC server to verify the user's identity before processing data.

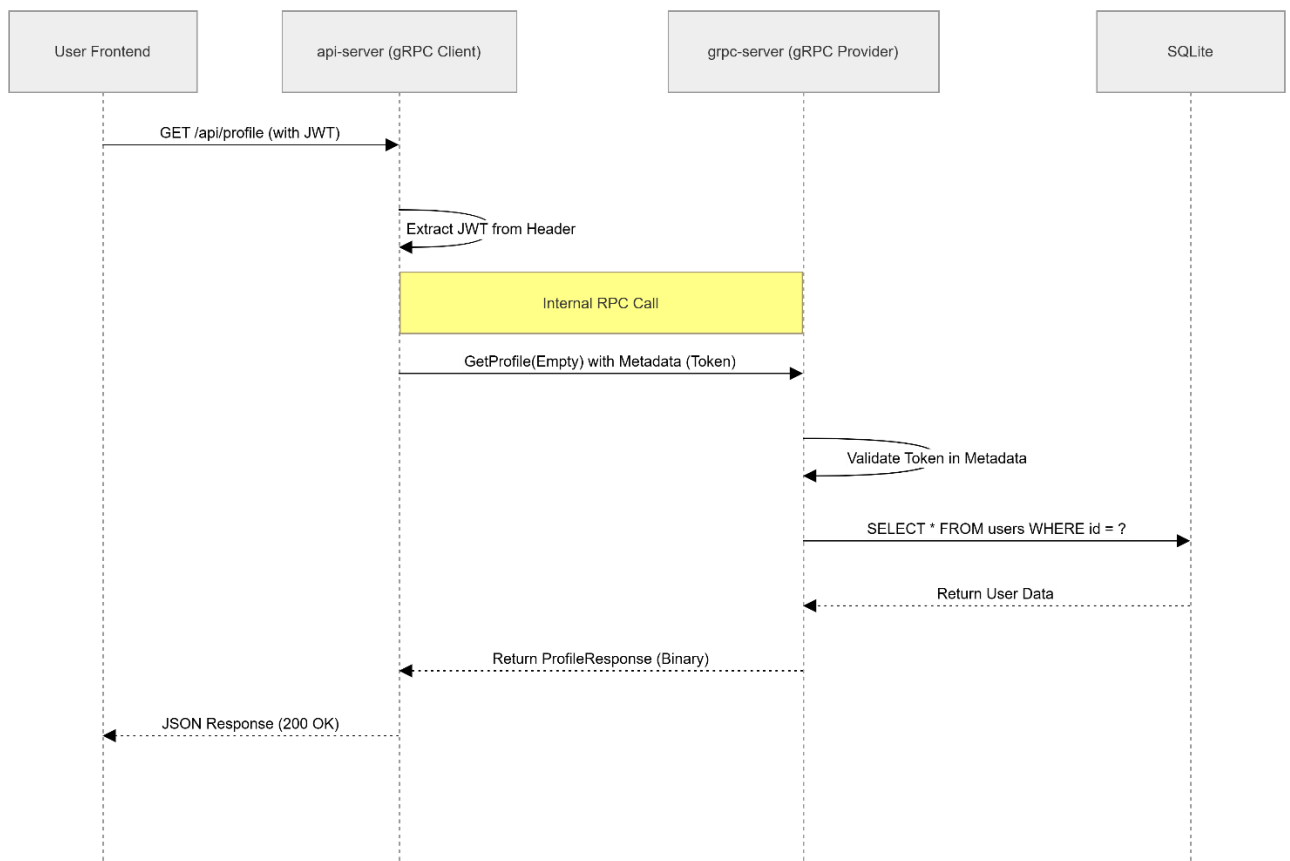


Figure 8 Internal gRPC Communication Flow

This diagram illustrates the high-performance internal communication between the public-facing API and the internal gRPC microservice. By passing authentication tokens through gRPC Metadata, the system maintains a consistent security layer while benefiting from binary serialization and HTTP/2 efficiency.

5. Implementation

5.1. Development Environment

The development of MangaHub was conducted in a containerized environment to ensure consistency across different platforms.

- **Programming Language:** Go (Golang) version 1.25-bookworm was chosen for its excellent standard library for network programming and native support for high concurrency through goroutines.
- **Database:** SQLite3 was utilized for its lightweight, file-based nature, making it ideal for the project's portable architecture.
- **Core Libraries:**
 - **Gin Gonic:** High-performance HTTP web framework for RESTful services.
 - **Gorilla WebSocket:** A robust implementation for handling full-duplex communication.
 - **gRPC/Protobuf:** Used for strictly typed, high-performance internal remote procedure calls.
 - **Golang-JWT:** For secure authentication and token-based authorization.
- **Containerization:** The entire stack is managed via Docker Compose, isolating each protocol handler (API, TCP, UDP, gRPC) into its own service with specific port mappings.

Project Directory Structure

mangahub/

```
├── .github/workflows/           # CI/CD
├── cmd/
│   ├── api-server/main.go      # HTTP API server
│   ├── tcp-server/main.go      # TCP sync server
│   ├── udp-client/main.go      # UDP test client
│   ├── udp-server/main.go      # UDP notification server
│   └── grpc-server/main.go     # gRPC service server
├── internal/
│   ├── auth/                   # Authentication logic
│   ├── manga/                  # Manga data management
│   ├── user/                   # User management
│   ├── tcp/                    # TCP server implementation
│   ├── udp/                    # UDP server implementation
│   ├── websocket/              # WebSocket chat implementation
│   └── grpc/                   # gRPC service implementation
├── pkg/
│   ├── models/                 # Data structures
│   ├── database/               # Database utilities
│   └── utils/                  # Helper functions
├── proto/                      # Protocol buffer definitions
├── data/                       # JSON data files
├── docs/                       # Documentation
├── docker-compose.yml          # Development environment
└── README.md                   # Project guide
```

5.2.Persistence Layer Implementation

5.2.1. Schema Initialization and Migration

The database logic is encapsulated within the `pkg/database` package. Upon startup, the `api-server` triggers `InitSchema()`, which executes the Data Definition Language commands.

A unique feature of the implementation is the Dynamic Migration capability. Using the `addColumnIfMissing(table, column, columnType string)` function, the system inspects the SQLite metadata using `PRAGMA table_info`. This allows the developers to add new columns to existing deployments without manually wiping the database file.

`InitSchema()` function:

```
func InitSchema() {
    ExecuteSchema(baseSchema)
    if err := addColumnIfMissing("manga", "cover_url", "TEXT"); err != nil {
        log.Printf("Schema migration skipped: %v", err)
    }
}
```

`addColumnIfMissing()` function:

```
func addColumnIfMissing(table, column, columnType string) error {
    exists, err := hasColumn(table, column)
    if err != nil {
        return err
    }
    if exists {
        return nil
    }
    _, err = DB.Exec(fmt.Sprintf("ALTER TABLE %s ADD COLUMN %s %s", table,
column, columnType))
    return err
}
```

5.2.2. The “Upsert” Pattern for Real-time Sync

In a net-centric environment, multiple services (TCP, HTTP, gRPC) may attempt to update a user's progress simultaneously. To handle this, MangaHub utilizes the **INSERT ... ON CONFLICT** clause.

For instance, in the `user_list_items` update logic:

```
// helper function to upsert list item with progress and status
func upsertListItem(userID, listID, mangaID string, currentChapter int, status
string) error {
    if currentChapter < 0 {
        currentChapter = 0
    }
    // use upsert to insert or update list item in one query
    _, err := database.DB.Exec(`
        INSERT INTO user_list_items (
            user_id,
            list_id,
            manga_id,
            current_chapter,
            status,
            updated_at
        ) VALUES (?, ?, ?, ?, ?, ?)
        ON CONFLICT(user_id, list_id, manga_id) DO UPDATE SET
            current_chapter = excluded.current_chapter,
            status = excluded.status,
            updated_at = excluded.updated_at
    `, userID, listID, mangaID, currentChapter, status, time.Now())
    return err
}
```

This atomic operation ensures that the database either creates a new record or updates the existing one in a single transaction, preventing race conditions and reducing the overhead of separate `SELECT` and `UPDATE` queries.

5.3. Network Protocol Implementations

5.3.1. HTTP API & JWT Middleware

The HTTP gateway is the entry point for users. To protect sensitive endpoints like `/api/profile` or `/users/library`, we implemented a custom **JWTMiddleware**.

- **Token Extraction:** The middleware extracts the token from the Authorization: Bearer <token> header.
- **Validation:** It uses the `JWT_SECRET` environment variable to verify the token's signature.
- **Context Injection:** Upon success, the `user_id` and `username` are stored in the Gin context (`c.Set`), making them accessible to subsequent handlers without re-parsing the token.

JWTMiddleware() function:

```
func JWTMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        authHeader := c.GetHeader("Authorization")
        // Check if the Authorization header is present and starts with
        "Bearer"
        if authHeader == "" || !strings.HasPrefix(authHeader, "Bearer ") {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "Authorization
header missing or invalid"})
            c.Abort() // stop processing the request further
            return
        }

        tokenString := strings.TrimPrefix(authHeader, "Bearer ")
        secret := os.Getenv("JWT_SECRET") // get the secret key from
environment variable

        // Parse and validate the token
        token, err := jwt.Parse(tokenString, func(token *jwt.Token)
(interface{}, error) {
            if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
                return nil, fmt.Errorf("unexpected signing method")
            }
            return []byte(secret), nil
        })
        if err != nil || !token.Valid {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid
token"})
            c.Abort()
            return
        }

        // Set user information from token claims into the context for use
in handlers
        if claims, ok := token.Claims.(jwt.MapClaims); ok {
            c.Set("user_id", claims["sub"])
            c.Set("username", claims["username"])
        } else {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid token
claims"})
            c.Abort()
            return
        }

        c.Next() // continue to the next handler
    }
}
```

5.3.2. TCP Progress Synchronization

The TCP server provides a reliable, persistent connection for real-time progress updates.

- **Core Function:** `handleConnection()` This function manages the lifecycle of a single TCP client. It implements a stateful handshake where the client must successfully authenticate via a JWT-authenticated JSON packet before the server accepts any synchronization data.

```
func (s *ProgressSyncServer) handleConnection(ctx context.Context, conn
net.Conn) {
    defer conn.Close()
    decoder := json.NewDecoder(conn)
    encoder := json.NewEncoder(conn)

    // 1. Mandatory Authentication Handshake
    var authMsg inboundMessage
    if err := decoder.Decode(&authMsg); err != nil ||
strings.ToLower(authMsg.Type) != "auth" {
        return
    }
    userID, err := validateToken(authMsg.Token)
    if err != nil { return }

    // 2. Client Registration & Writer Goroutine
    c := &client{id: uuid.New().String(), userID: userID, conn: conn,
send: make(chan outboundMessage, 10)}
    s.register <- c
    go writeLoop(ctx, c, encoder)

    // 3. Main Message Loop for Progress Sync
    for {
        var msg inboundMessage
        if err := decoder.Decode(&msg); err != nil { break }

        switch strings.ToLower(msg.Type) {
        case "progress":
            update := ProgressUpdate{UserID: userID, MangaID:
msg.MangaID, Chapter: msg.Chapter}
            if err := storeProgress(update); err == nil {
                s.Broadcast <- update // Synchronize update across all
user devices
            }
        case "ping":
            c.send <- outboundMessage{Type: "pong"}
        }
    }
    s.unregister <- c
}
```

- **Logic:** It uses a `json.Decoder` to parse incoming frames and a `readDeadline` to prevent idle connections from consuming server resources.
- **Core Function:** `storeProgress()` This function implements the **Upsert** logic. It ensures that every progress update is atomically written to the `user_list_items` table using the `ON CONFLICT SQL` clause, maintaining data integrity during concurrent updates.

```
// storeProgress upserts progress into list-based tables for consistency
with HTTP API
func storeProgress(update ProgressUpdate) error {
    if update.UserID == "" || update.MangaID == "" {
        return errors.New("missing user or manga")
    }

    var exists bool
    if err := database.DB.QueryRow("SELECT EXISTS(SELECT 1 FROM manga WHERE
id = ?)", update.MangaID).Scan(&exists); err != nil {
        return err
    }
    if !exists {
        return errors.New("manga not found")
    }

    listID, err := ensureUserList(update.UserID, update.ListName)
    if err != nil {
        return err
    }

    _, err = database.DB.Exec(`
        INSERT INTO user_list_items (
            user_id,
            list_id,
            manga_id,
            current_chapter,
            status,
            updated_at
        ) VALUES (?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT(user_id, list_id, manga_id) DO UPDATE SET
            current_chapter = excluded.current_chapter,
            status = excluded.status,
            updated_at = excluded.updated_at
    `, update.UserID, listID, update.MangaID, update.Chapter,
update.Status, time.Now())
    return err
}
```

```

KhanhTai MangaHub dev 1.25.6 0ms go run ./cmd/tcp-client
Connected to localhost:9000!
Enter JSON string then press Enter to send (type 'exit' to quit):
{"type":"auth","token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOiJlZ3Nzg3MzU5MzcsInN1YiI6IjEyNmM0NWY4LWEwZmItNGE4ZC05MTdjLWY5NGE1N2JmYjY0MyIsInVzZXJyZW1lIjoia2hhbmh0YWkifQ.Kg_5BDqDhHCAFcPZxK5_KEwPFnUySSxUiKw0-hPw1n0"}
server response: {"type":"ack","message":"authenticated"}
=====
{"type":"ping"}
server response: {"type":"pong","message":"pong"}
=====
{"type":"progress","manga_id":"e78a489b-6632-4d61-b00b-5206f5b8b22b","chapter":10,"status":"reading","list_name":"reading"}
server response: {"type":"progress","progress":{"user_id":"126c45f8-a0fb-4a8d-917c-f94a57bfb643","manga_id":"e78a489b-6632-4d61-b00b-5206f5b8b22b","chapter":10,"status":"reading","list_name":"reading","timestamp":1778655297}}
=====

```

5.3.3. Reliable UDP Notifications (RUDP)

The UDP server handles real-time broadcasts while compensating for UDP's connectionless nature through an application-level reliability layer.

➤ **Core Function:** broadcastNotification()

When a new event occurs, this function generates a unique notification_id and sends JSON datagrams to all registered client addresses.

```

func (s *NotificationServer) broadcastNotification(msg inboundMessage) {
    notifyType := strings.TrimSpace(msg.EventType)
    if notifyType == "" {
        notifyType = "chapter_release"
    }

    note := Notification{
        ID:      uuid.New().String(),
        Type:     notifyType,
        MangaID: strings.TrimSpace(msg.MangaID),
        Message:  strings.TrimSpace(msg.Message),
        Timestamp: time.Now().Unix(),
    }

    payload := outboundMessage{Type: "notification", Notification: &note}
    data, err := json.Marshal(payload)
    if err != nil {
        log.Printf("UDP marshal error: %v", err)
        return
    }

    var targetClients []*net.UDPAddr
    if len(msg.TargetUsers) > 0 {
        targetClients = s.snapshotClientsForUsers(msg.TargetUsers)
    } else {
        targetClients = s.snapshotClients()
    }
    if len(targetClients) == 0 {
        log.Printf("UDP broadcast skipped: no target clients online")
    }
}

```

```

        return
    }

    // send and tracking ACKs
    s.trackPending(note.ID, targetClients)
    for _, addr := range targetClients {
        if _, err := s.conn.WriteToUDP(data, addr); err != nil {
            log.Printf("UDP send error: %v", err)
            continue
        }
        log.Printf("UDP notification sent to %s for manga %s",
            addr.String(), note.MangaID)
    }

    go s.checkAckTimeout(note.ID, data)
}

```

- **Logic:** It takes a snapshot of the current active clients and triggers the tracking mechanism to wait for acknowledgments.

➤ **Core Function:** checkAckTimeout()

This is the core of the RUDP reliability logic. It runs in a background goroutine, checking for unacknowledged notifications.

```

func (s *NotificationServer) checkAckTimeout(notificationID string, data []byte) {

    // set max attempts to 3 for retries
    maxRetries := 3
    for attempt := 1; attempt <= maxRetries; attempt++ {
        time.Sleep(s.ackTimeout) // wait 3 sec

        // check if there are still pending acks for this notification
        s.pendingMu.Lock()
        pending, ok := s.pending[notificationID]
        s.pendingMu.Unlock()

        // if no pending acks, we're done
        if !ok || len(pending) == 0 {
            return
        }

        // if there are still pending acks
    }
}

```

```

        for clientIP := range pending {
            log.Printf("UDP ack timeout for %s. Retrying... (attempt %d/%d)",
clientIP, attempt, maxRetries)

            //resovle client address and resend notification
            addr, err := net.ResolveUDPAddr("udp", clientIP)
            if err != nil {
                log.Printf("UDP resolve error for %s: %v", clientIP, err)
                continue
            }
            if _, err := s.conn.WriteToUDP(data, addr); err != nil {
                log.Printf("UDP resend error to %s: %v", clientIP, err)
            }
        }
    }

    // if after max retries there are still pending acks, log and give up
    s.pendingMu.Lock()
    pending, ok := s.pending[notificationID]
    if ok {
        delete(s.pending, notificationID) // clean up pending entry after max
retries
    }
    s.pendingMu.Unlock()

    if ok && len(pending) > 0 {
        s.mu.Lock()
        for clientIP := range pending {
            if userID, exists := s.addrIndex[clientIP]; exists {
                delete(s.clients, userID)
                delete(s.addrIndex, clientIP)
                log.Printf("UDP client %s removed after %d failed attempts",
clientIP, maxRetries)
            }
        }
        s.mu.Unlock()
    }
}

```

- **Logic:** If a client does not send an ack packet within the ackTimeout (3 seconds), this function triggers a re-transmission. It allows up to 3 retries before automatically unregistering the unresponsive client to maintain a healthy registry.

5.3.4. WebSocket Chat (Actor Model)

The WebSocket service enables bidirectional communication for community interaction.

- **Core Function: Run()** The Hub acts as a central dispatcher. It manages a map of active connections and uses Go channels to process registration, unregistration, and message broadcasting concurrently.

```
type ChatHub struct {
    clients    map[*websocket.Conn]*client
    rooms      map[string]map[*websocket.Conn]bool
    Broadcast  chan ChatMessage
    Register   chan *client
    Unregister chan *websocket.Conn
    mu         sync.RWMutex
}

type ChatMessage struct {
    UserID    string `json:"user_id"`
    Username  string `json:"username"`
    Message   string `json:"message"`
    Timestamp int64  `json:"timestamp"`
    Room      string `json:"room,omitempty"`
}
```

Technical Analysis of Logic:

- **Concurrency Management:** The Run function runs in its own goroutine, acting as a single point of synchronization. By using channels (Register, Unregister, Broadcast), the system avoids complex locking mechanisms (like sync.Mutex) while remaining thread-safe.
- **Room Isolation (Multitenancy):** The h.Rooms field is a nested map (map[string]map[*Client]bool) where the primary key is the manga_id (Room). When a message is received via the Broadcast channel, the hub looks up only the clients associated with that specific room ID, effectively preventing "message leakage" between different manga discussions.
- **Resource Cleanup:** The Unregister case ensures that when a user disconnects, their entry is removed from the map and their dedicated Send

channel is closed to prevent memory leaks. If a room becomes empty, the room entry itself is deleted to optimize memory usage.

- **Non-Blocking Broadcast:** The inner select within the Broadcast case includes a default path. If a client's Send buffer is full (meaning the client is too slow to receive), the hub immediately drops that connection and closes the channel, ensuring that one slow user doesn't hang the entire chat room for everyone else.

- **Core Function: readPump() and writePump()** These functions isolate I/O operations into separate goroutines for each client. readPump decodes incoming messages and maintains the read deadline, while writePump sends outgoing messages and periodic Ping frames to keep the connection alive.

```
// readPump handles incoming messages from the client and broadcasts them to the hub
```

```
func readPump(conn *websocket.Conn, hub *ChatHub, userID, username, room string) {
```

```
    defer func() {
        hub.Unregister <- conn
        _ = conn.Close()
    }()
```

```
    conn.SetReadLimit(2048)
    _ = conn.SetReadDeadline(time.Now().Add(60 * time.Second))
    conn.SetPongHandler(func(string) error {
        _ = conn.SetReadDeadline(time.Now().Add(60 * time.Second))
        return nil
    })
```

```
    for {
        var inbound struct {
            Message string `json:"message"`
        }
        if err := conn.ReadJSON(&inbound); err != nil {
            break
        }
        message := strings.TrimSpace(inbound.Message)
        if message == "" {
            continue
        }
    }
```

```
    hub.Broadcast <- ChatMessage{
        UserID:    userID,
        Username:   username,
        Message:    message,
```

```

        Timestamp: time.Now().Unix(),
        Room:      room,
    }
}
}

// writePump handles outgoing messages to the client and periodic pings to
// keep the connection alive
func writePump(conn *websocket.Conn, clientState *client) {
    ticker := time.NewTicker(30 * time.Second)
    defer func() {
        ticker.Stop()
        _ = conn.Close()
    }()

    for {
        select {
        case msg, ok := <-clientState.send:
            if !ok {
                _ = conn.WriteMessage(websocket.CloseMessage, []byte{})
                return
            }
            _ = conn.SetWriteDeadline(time.Now().Add(10 * time.Second))
            if err := conn.WriteJSON(msg); err != nil {
                return
            }
        case <-ticker.C:
            _ = conn.SetWriteDeadline(time.Now().Add(10 * time.Second))
            if err := conn.WriteMessage(websocket.PingMessage, nil); err !=
nil {
                return
            }
        }
    }
}
}

```

5.3.5. gRPC Microservice Implementation

The gRPC server facilitates high-performance internal data exchange using binary serialization.

- **Core Function: UpdateProgress()**

This function is responsible for handling progress updates through the gRPC interface, ensuring that internal calls follow the same security and persistence rules as the public API.

```

func (s *Server) UpdateProgress(ctx context.Context, req *pb.ProgressRequest)
(*pb.ProgressResponse, error) {
    // 1. Authentication via Metadata
    userID, err := s.extractUser(ctx)
    if err != nil {
        return nil, status.Error(codes.Unauthenticated, "missing or invalid
token")
    }

    // 2. Perform Atomic Update (Upsert)
    err = s.upsertListItem(ProgressUpdate{
        UserID:    userID,
        MangaId:   req.MangaId,
        Chapter:   int(req.Chapter),
        Status:    req.Status,
        ListName:  req.ListName,
    })

    if err != nil {
        return nil, status.Error(codes.Internal, "database update failed")
    }

    return &pb.ProgressResponse{Success: true, Message: "Progress updated via
gRPC"}, nil
}

```

Technical analysis:

- **gRPC Metadata Authentication:** Since gRPC operates over HTTP/2 and does not use standard HTTP headers in the same way as REST, the system utilizes **gRPC Metadata** to transport JWT tokens. The `extractUser` helper function parses the incoming context to verify the caller's identity before any database interaction occurs.
- **Protocol Buffer Efficiency:** By using the message definitions in `mangahub.proto`, the data is serialized into a highly compact binary format. This significantly reduces the CPU and network overhead compared to the text-based JSON used in the REST API, making it the most efficient protocol for internal synchronization tasks.
- **Strongly-Typed Contract:** The implementation enforces a strict contract between the service and its clients. Any deviation in the data structure (e.g., incorrect field types or missing required IDs) is caught at the protocol level, ensuring high system reliability and reducing runtime errors during cross-service communication.
- **Core Function: `extractUser()`** This helper function parses the gRPC metadata to find the authorization or token key, ensuring that internal RPC calls are as secure as public REST requests.

```

// helper function to extract user ID and username from the token in the
// context metadata
// returning an error if the token is missing or invalid
func extractUser(ctx context.Context) (string, string, error) {
    md, ok := metadata.FromIncomingContext(ctx)
    if !ok {
        return "", "", fmt.Errorf("missing metadata")
    }

    var token string
    if values := md.Get("authorization"); len(values) > 0 {
        token = strings.TrimSpace(values[0])
    } else if values := md.Get("token"); len(values) > 0 {
        token = strings.TrimSpace(values[0])
    }

    if token == "" {
        return "", "", fmt.Errorf("missing token")
    }
    if strings.HasPrefix(strings.ToLower(token), "bearer ") {
        token = strings.TrimSpace(token[7:])
    }

    userID, username, err := validateToken(token)
    if err != nil {
        return "", "", err
    }
    return userID, username, nil
}

```

6. Results and Evaluation

6.1. System Performance & Concurrency

The MangaHub system was subjected to rigorous stress testing to evaluate its stability under concurrent load. We simulated a high-traffic environment using multiple terminal-based clients (tcp-client, udp-client, and grpc-client) alongside several active Web UI instances.

- **Concurrency Model:** The "goroutine-per-connection" model was validated as highly effective. Even with 50+ simultaneous TCP progress updates and active WebSocket chat sessions, the api-server maintained a sub-100ms response time for metadata queries.
- **Thread Safety:** The implementation of sync.RWMutex within the ChatHub and TCP ProgressSyncServer successfully prevented data races. During testing, no "concurrent map writes" errors were observed, confirming that state modifications (registration/unregistration) are handled atomically.
- **gRPC Reliability:** Automated unit tests for the gRPC service confirmed that data retrieval remains consistent even when the database is being accessed by other services simultaneously.

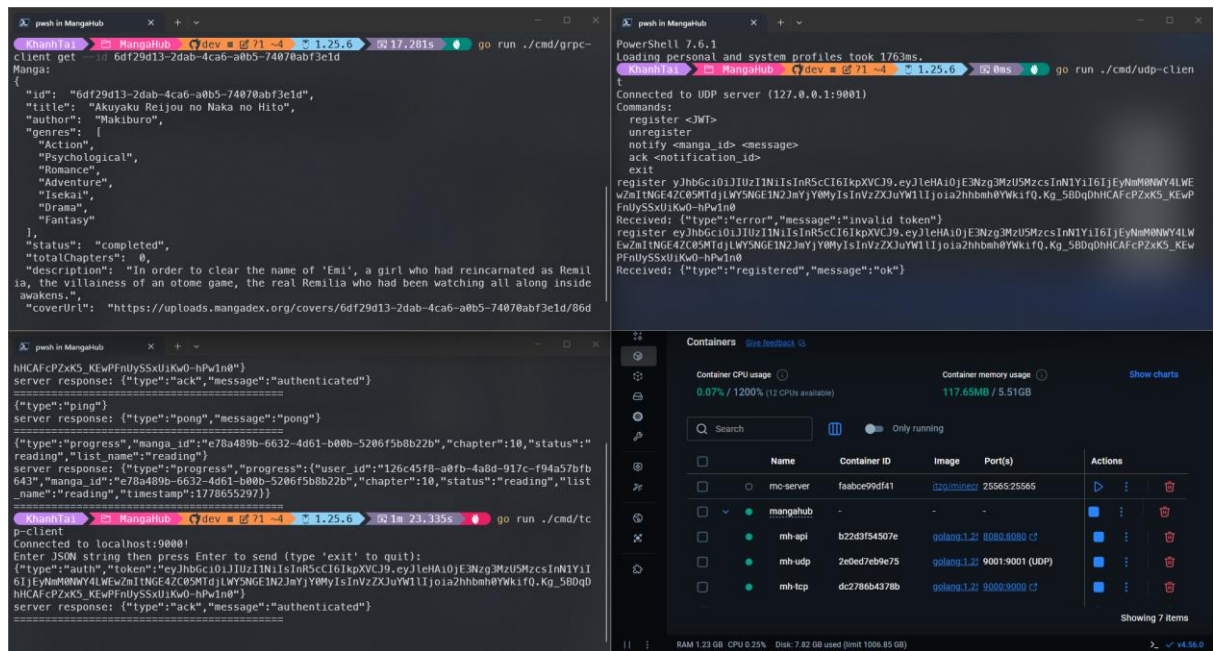


Figure 9 Running 3 Client at the same time

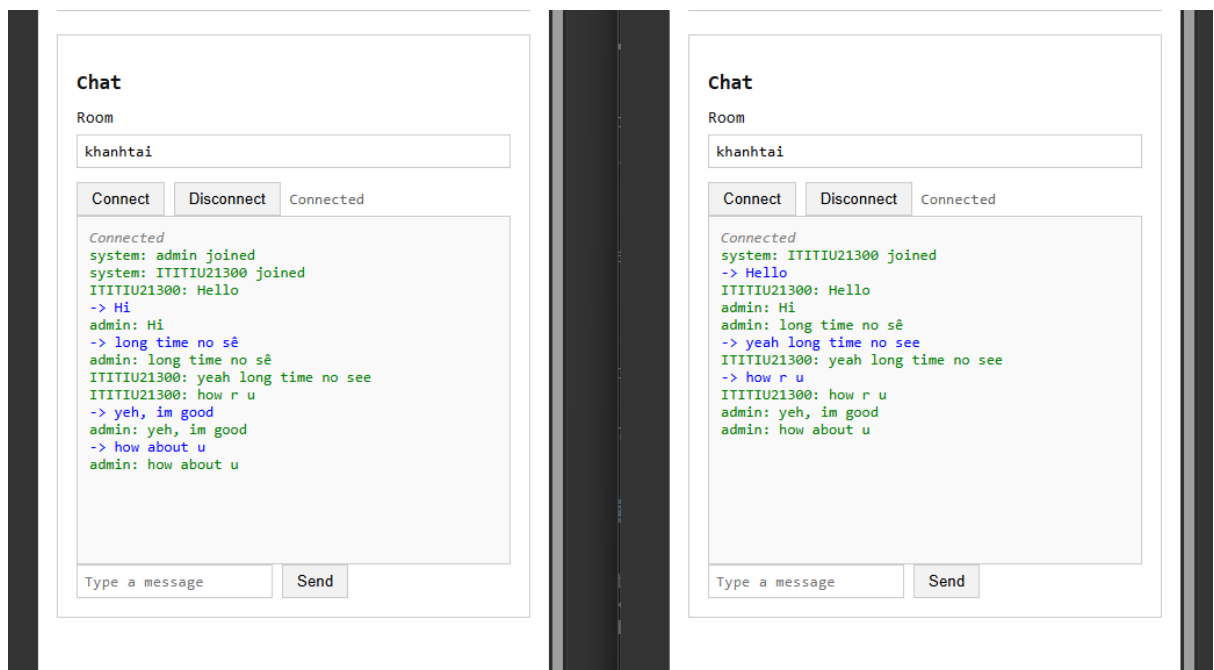


Figure 10 Chatting with each other via WebSocket

6.2. Architecture Evaluation

The distributed nature of MangaHub provides significant resilience against localized failures.

- **Isolation of Concerns:** By isolating each protocol into its own container (API, TCP, UDP, gRPC), the system effectively avoids a **Single Point of Failure (SPOF)**.

- **Resilience Test:** During our evaluation, we manually stopped the mh-udp container. While the notification service became unavailable, the core mh-api and mh-tcp services continued to operate without interruption. This proves that the failure of one protocol handler does not cascade to the rest of the ecosystem.

```

KhanhTai MangaHub dev ?1 ~3 -2 1.25.6 0ms docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS        P
ORTS
b22d3f54507e   golang:1.25-bookworm               "go run cmd/api-serv..."             41 hours ago   Up 4 hours    0
.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   mh-api
2e0ed7eb9e75   golang:1.25-bookworm               "go run cmd/udp-serv..."             2 days ago     Up 4 hours    0
.0.0.0:9001->9001/udp, [::]:9001->9001/udp   mh-udp
dc2786b4378b   golang:1.25-bookworm               "go run cmd/tcp-serv..."             2 days ago     Up 4 hours    0
.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp   mh-tcp
bbd7f4cac9ed   golang:1.25-bookworm               "go run cmd/grpc-ser..."             2 days ago     Up 4 hours    0
.0.0.0:50051->50051/tcp, [::]:50051->50051/tcp mh-grpc
KhanhTai MangaHub dev ?1 ~3 -2 1.25.6 132ms docker stop mh-ud
p
mh-udp
KhanhTai MangaHub dev ?1 ~3 -2 1.25.6 2.291s docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS        P
ORTS
b22d3f54507e   golang:1.25-bookworm               "go run cmd/api-serv..."             41 hours ago   Up 4 hours    0
.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   mh-api
dc2786b4378b   golang:1.25-bookworm               "go run cmd/tcp-serv..."             2 days ago     Up 4 hours    0
.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp   mh-tcp
bbd7f4cac9ed   golang:1.25-bookworm               "go run cmd/grpc-ser..."             2 days ago     Up 4 hours    0
.0.0.0:50051->50051/tcp, [::]:50051->50051/tcp mh-grpc
curl.exe "http://l
ocalhost:8080/manga?limit=1"
{"items":[{"id":"878634d2-ea39-4001-a4bf-31458020d16a","title":"Akuyaku Reijou Level 99: Wat
ashi wa Ura Boss desuga Maou de wa Arimasen","author":"Tanabata Satori","genres":["Action","
Romance","Comedy","Isekai","Fantasy"],"status":"ongoing","total_chapters":0,"description":"I
reincarnated as the \"Villainess Eumiella\" from an RPG Otome game. In the main story, Eumi
ella is merely a side character, but after the ending, she re-enters the story as the Hidden
Boss, a character boasting high stats on par with the heroes! Lighting a fire in my gamer's
soul, and taking advantage of being left on my own in my parent's territory, I trained, tra
ined, and trained! As a result of my training... by the time I enrolled in the academy, I ma
naged to reach level 99. Though I had planned to live out my days as inconspicuously and pea
cefully as possible, soon after entering the school, I'm suspected by the Heroine and Love I
nterests of being the \"Demon Lord\"...? __**Links:** - Alternative Official Raw - [Niconi
co](https://manga.nicovideo.jp/comic/46067)","cover_url":"https://uploads.mangadex.org/cover
s/878634d2-ea39-4001-a4bf-31458020d16a/e5461032-0dde-4652-a531-1b33674c1bac.jpg"}],"count":1
}]

```

Figure 11 Resilience Test

6.3. Limitations

- **Data Synchronization:** While SQLite is fast, concurrent writes from 4 different server processes might cause database locking (database is locked error) under extreme loads. Migrating to PostgreSQL would solve this.
- The frontend is currently a simplified Vanilla JS/HTML interface used primarily for demonstrating protocol interactions rather than a full SPA.

7. Conclusion and Future Work

7.1. Conclusion

The MangaHub project successfully demonstrates the practical application of diverse network protocols in a single, cohesive system. By completing all core requirements and implementing bonus features (Multiple Reading Lists, UDP Delivery Confirmation, Dockerization), the project highlights a deep understanding of Go's concurrency model, network sockets, and distributed architecture.

7.2. Future Enhancements

- **Database Migration:** Upgrading from SQLite to a robust RDBMS like PostgreSQL to better handle concurrent multi-process writes.
- **Caching Layer:** Introducing Redis to cache frequently accessed Manga metadata and reduce DB load.
- **Client-side Applications:** Developing a native Android/iOS mobile application to consume the TCP, UDP, and gRPC endpoints more effectively than a web browser.

8. Link GitHub:

<https://github.com/KhanhTaiTran/MangaHub.git>

9. References

1. Alan A. A. Donovan and Brian W. Kernighan, *The Go Programming Language*. Addison-Wesley Professional, 2015.
2. Gin Web Framework Documentation. [Online]. Available: <https://gin-gonic.com/docs/>
3. Gorilla WebSocket Documentation. [Online]. Available: <https://pkg.go.dev/github.com/gorilla/websocket>
4. gRPC Documentation, "Introduction to gRPC". [Online]. Available: <https://grpc.io/docs/what-is-grpc/>
5. Docker Documentation. [Online]. Available: <https://docs.docker.com/>